# Machine Learning

## Chapitre 3: Régression

Mme BEDAD Fatima
fatima.bedad@univ-temouchent.edu.dz

2023-2024

# Table of Contents

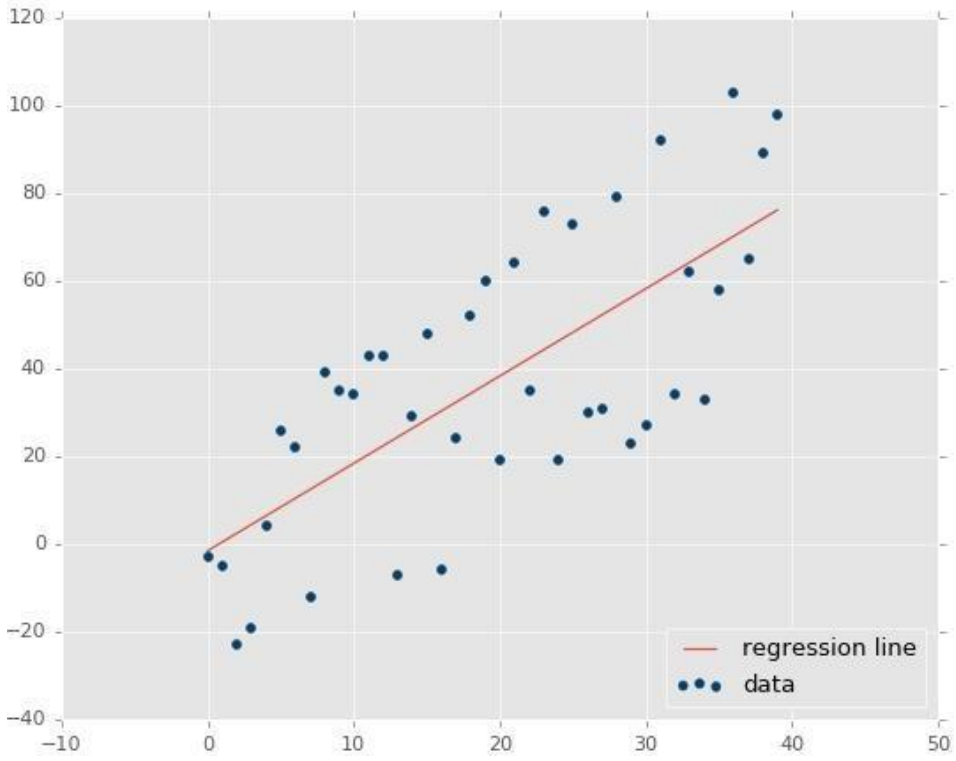
1) Dataset Loading
2) Data Preprocessing + Exploratory Data Analysis
3) Feature selection
4) Dimensionality Reduction
5) Training and Test Sets
6) Supervised learning
7) Unsupervised learning
8) Save and Load Machine Learning Models

# Supervised learning

# Regression

# 1. Regression: Linear regression

**Linear regression** performs the task to predict a dependent variable value (**y**) based on a given independent variable (**x**). So, this regression technique **finds out a linear relationship** between **x** (input) and **y** (output).




**The equation of the above line is :**

**Y= aX + b**

**A regression model involving multiple variables can be represented as:**

**Y = a1X1 + a2X2 + ... + anXn +b**

# 1. Regression: Linear regression

**Linear regression** refers to a model that assumes a linear relationship between **input** variables and the **target** variable.

With a **single input** variable, this relationship is a **line**, and with **higher dimensions**, this relationship can be thought of as a **hyperplane** that connects the input variables to the target variable. The **coefficients** of the model are found via an **optimization process** that seeks to minimize the sum squared error between the predictions (yp) and the expected target values (**y**).

$$loss = \sum_{i=0}^{n} (y_i - yp_i)^2$$

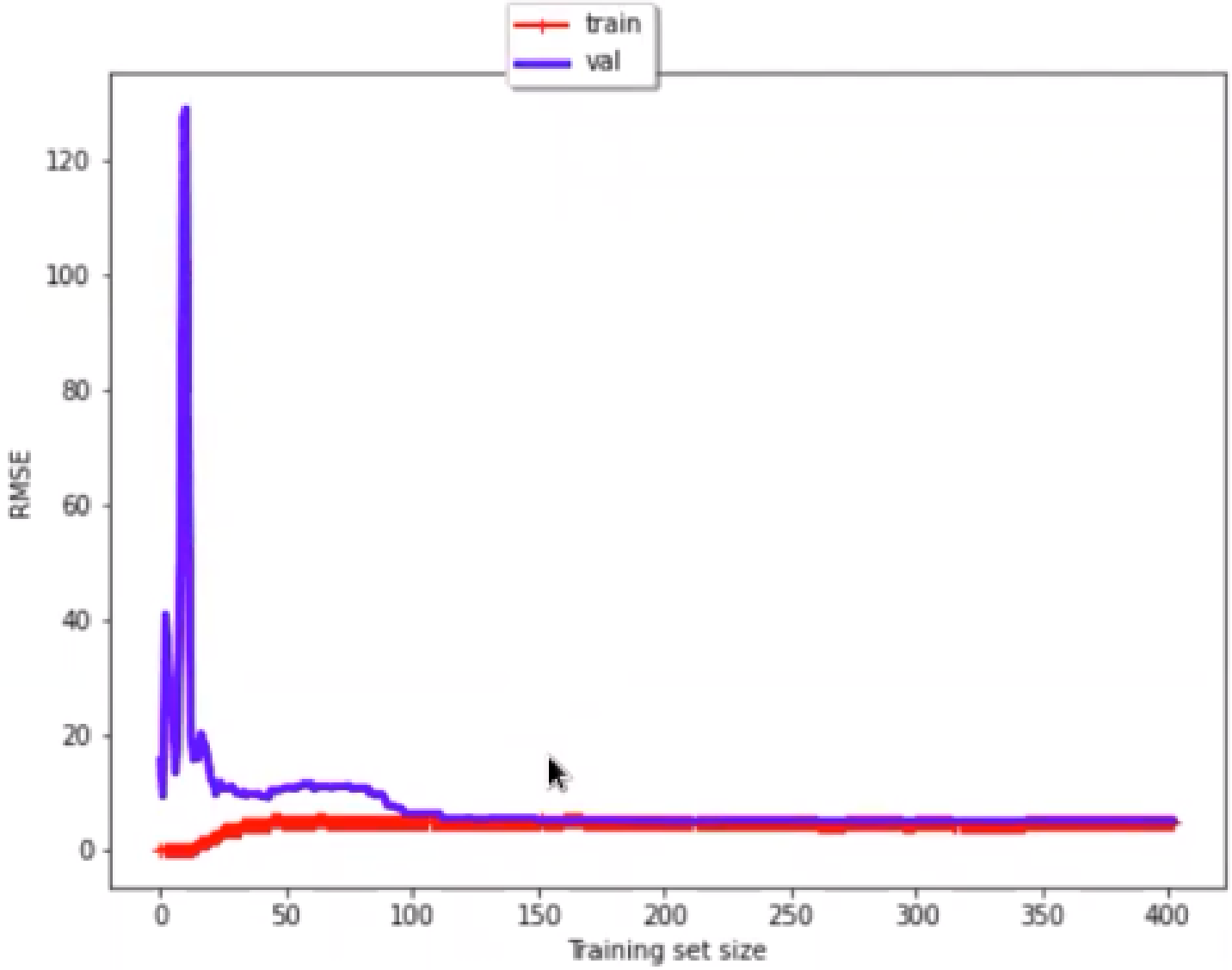
```
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn import datasets, linear_model
from sklearn import metrics
import numpy as np

dat = datasets.load_boston()
X = dat.data
Y = dat.target
print("Examples = ",X.shape ," Labels = ", Y.shape)

fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(X[:,0], Y, edgecolors=(0, 0, 0))
ax.plot([Y.min(), Y.max()], [Y.min(), Y.max()], 'k--', lw=4)
ax.set_xlabel('F')
ax.set_ylabel('Y')
plt.show()

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0], X[:, 1], Y, c='b', marker='o',cmap=plt.cm.Set1, edgecolor='k', s=40)
ax.set_title("My Data")
ax.set_xlabel("F1")
ax.w_xaxis.set_ticklabels([])
ax.set_ylabel("F2")
ax.w_yaxis.set_ticklabels([])
ax.set_zlabel("Y")
ax.w_zaxis.set_ticklabels([])
plt.show()

X_train, X_test, Y_train, Y_test = train_test_split(X,
            Y, test_size= 0.20, random_state=100)
print("X_train = ",X_train.shape ," Y_test = ", Y_test.shape)
regressor = linear_model.LinearRegression()
regressor.fit(X_train, Y_train)
predicted = regressor.predict(X_test)
```

```
import pandas as pd
df = pd.DataFrame({'Actual': Y_test.flatten(), 'Predicted': predicted.flatten()})
print(df)

df1 = df.head(25)
df1.plot(kind='bar',figsize=(12,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()

predicted_all = regressor.predict(X)
fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(X[:,0], Y, edgecolors=(0, 0, 1))
ax.scatter(X[:,0], predicted_all, edgecolors=(1, 0, 0))
ax.set_xlabel('Measured')
ax.set_ylabel('Predicted')
plt.show()

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0], X[:, 1], Y, c='b', marker='o',cmap=plt.cm.Set1, edgecolor='k', s=40)
ax.scatter(X[:, 0], X[:, 1], predicted_all, c='r', marker='o',cmap=plt.cm.Set1, edgecolor='k',
s=40)
ax.set_title("My Data")
ax.set_xlabel("F1")
ax.w_xaxis.set_ticklabels([])
ax.set_ylabel("F2")
ax.w_yaxis.set_ticklabels([])
ax.set_zlabel("Y")
ax.w_zaxis.set_ticklabels([])
plt.show()

print('Mean Absolute Error     : ', metrics.mean_absolute_error(Y_test, predicted))
print('Mean Squared Error      : ', metrics.mean_squared_error(Y_test, predicted))
print('Root Mean Squared Error: ', np.sqrt(metrics.mean_squared_error(Y_test, predicted)))
```

# 1. Regression: Learning Curves

```
def plot_learning_curves(model, X, y):
  from sklearn.metrics import mean_squared_error
  X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)
  train_errors, val_errors = [], []
  for m in range(1, len(X_train)):
    model.fit(X_train[:m], y_train[:m])
    y_train_predict = model.predict(X_train[:m])
    y_val_predict = model.predict(X_val)
    train_errors.append(mean_squared_error(y_train_predict, y_train[:m]))
    val_errors.append(mean_squared_error(y_val_predict, y_val))

  fig, ax = plt.subplots(figsize=(12,8))
  ax.plot(np.sqrt(train_errors), "r-+", linewidth=2, label="train")
  ax.plot(np.sqrt(val_errors), "b-", linewidth=3, label="val")
  ax.legend(loc='upper right', bbox_to_anchor=(0.5, 1.1),ncol=1, fancybox=True, shadow=True)
  ax.set_xlabel('Training set size')
  ax.set_ylabel('RMSE')
  plt.show()
```

# 1. Regression: Learning Curves



# 1. Regression: Ridge regression

- A problem with linear regression is that estimated **coefficients** of the model can become **large**, making the model **sensitive** to inputs and possibly **unstable**. This is particularly true for problems with **few samples (n)** than input **predictors (p)** or variables (so-called **p >> n** problems).

- One approach to address the stability of regression models is to **change the loss function** to include **additional costs** for a model that has large coefficients. Linear regression models that use these modified loss functions during training are referred to collectively as **penalized linear regression**.

- One popular penalty is to penalize a model based on **the sum of the squared coefficient values**. This is called an **L2 penalty**.

# 1. Regression: Ridge regression

**Ridge regression addresses some of the problems of Ordinary Least Squares by imposing a penalty on the size of the coefficients.**

```
from sklearn import datasets, linear_model

# Regularization strength; must be a positive float. Regularization improves the conditioning of the problem and reduces the variance of the estimates. Larger values specify stronger regularization.

regressor = linear_model.Ridge(alpha=.5)
regressor.fit(X_train, Y_train)
predicted = regressor.predict(X_test)

# Bayesian Ridge Regression
regressor = linear_model.BayesianRidge()

# Lasso
regressor  = linear_model.Lasso(alpha=0.1)
```

# 1. Regression: Kernel Ridge regression

In order to explore nonlinear relations of the regression problem

```
from sklearn.kernel_ridge import KernelRidge

# kernel = [linear, polynomial, rbf]
regressor = KernelRidge(kernel ='rbf', alpha=1.0)
regressor.fit(X_train, Y_train)
predicted = regressor.predict(X_test)
```

# 1. Regression: Polynomial Regression

## How to use a linear model to fit nonlinear data ?

A simple way to do this is to add powers of each feature as new features, then train a linear model on this extended set of features. This technique is called ***Polynomial Regression***.

# 1. Regression: Polynomial Regression

```
# generate some nonlinear data
import numpy as np
m = 1000
X = 6 * np.random.rand(m, 1) - 3
Y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
print("Examples = ",X.shape ," Labels = ", Y.shape)

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(X[:,0], Y, edgecolors=(0, 0, 1))
ax.set_xlabel('F')
ax.set_ylabel('Y')
plt.show()

from sklearn.preprocessing import PolynomialFeatures
poly_features = PolynomialFeatures(degree=2,
include_bias=False)
X_poly = poly_features.fit_transform(X)
print("Examples = ",X_poly.shape ," Labels = ", Y.shape)

from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_poly,
            Y, test_size= 0.20, random_state=100)
```

```
from sklearn import linear_model
regressor = linear_model.LinearRegression()
regressor.fit(X_train, Y_train)
predicted = regressor.predict(X_poly)

fig, ax = plt.subplots(figsize=(12,8))
ax.scatter(X, Y, edgecolors=(0, 0, 1))
ax.scatter(X,predicted,edgecolors=(1, 0, 0))
ax.set_xlabel('F')
ax.set_ylabel('Y')
plt.show()

B = regressor.intercept_
A = regressor.coef_
print(A)
print(B)
print("The model estimates : Y = ",B[0]," + ",A[0,0]," X + ",A[0,1]," X^2")

from sklearn import metrics
predicted = regressor.predict(X_test)
print('Mean Absolute Error     : ', metrics.mean_absolute_error(Y_test, predicted))
print('Mean Squared Error      : ', metrics.mean_squared_error(Y_test, predicted))
print('Root Mean Squared Error: ', np.sqrt(metrics.mean_squared_error(Y_test,
predicted)))
```

# 1. Regression: Support Vector Regression SVR

**The Support Vector Regression (SVR) uses the same principles as the SVM for classification, with only a few minor differences.**

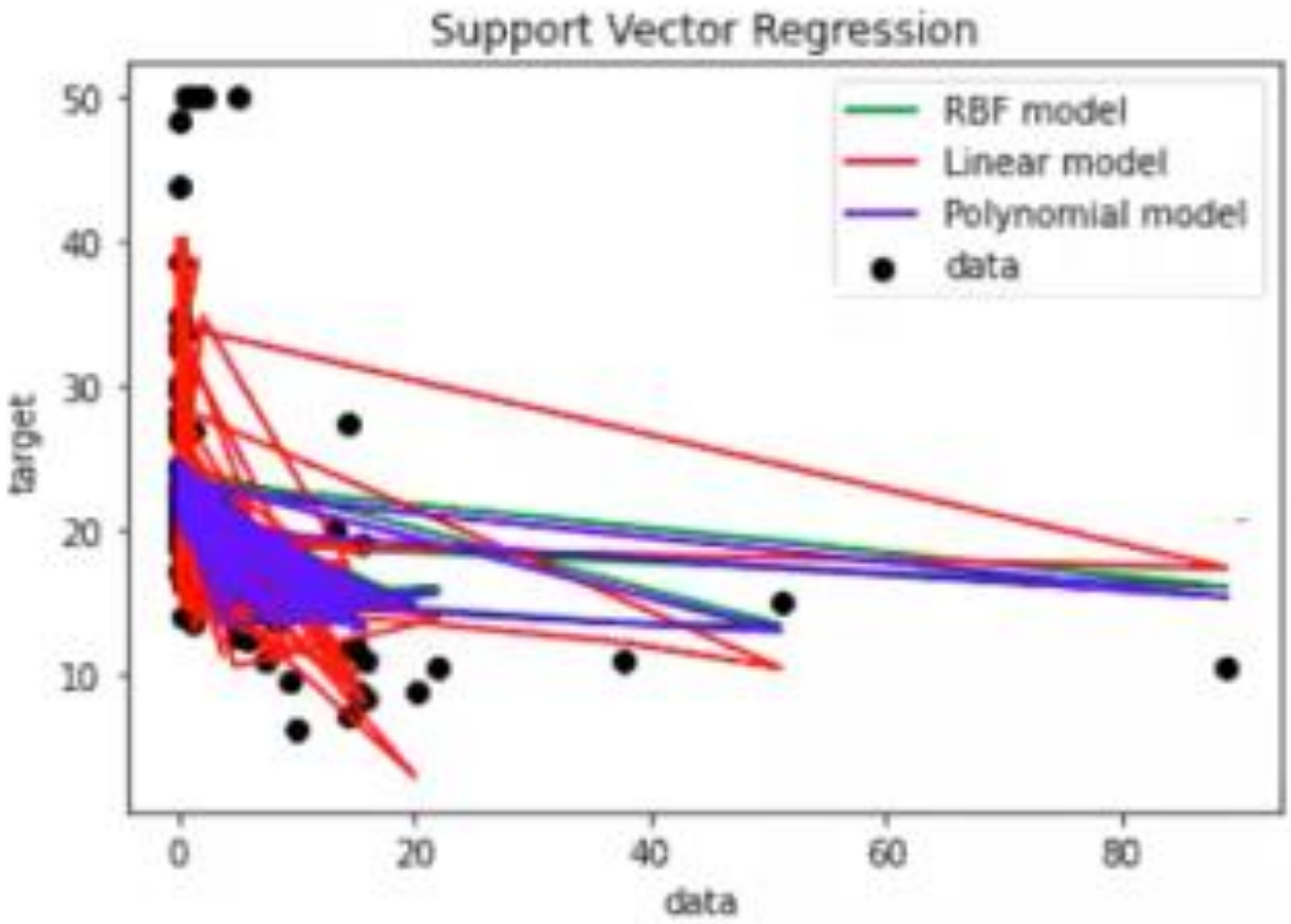
```
from sklearn.svm import SVR

svr_rbf  = SVR(kernel='rbf', C=100, gamma=0.1)

svr_lin  = SVR(kernel='linear', C=100, gamma=0.1)

svr_poly = SVR(kernel='poly', C=100, gamma=0.1, degree=3)
```

# 1. Regression: Support Vector Regression SVR

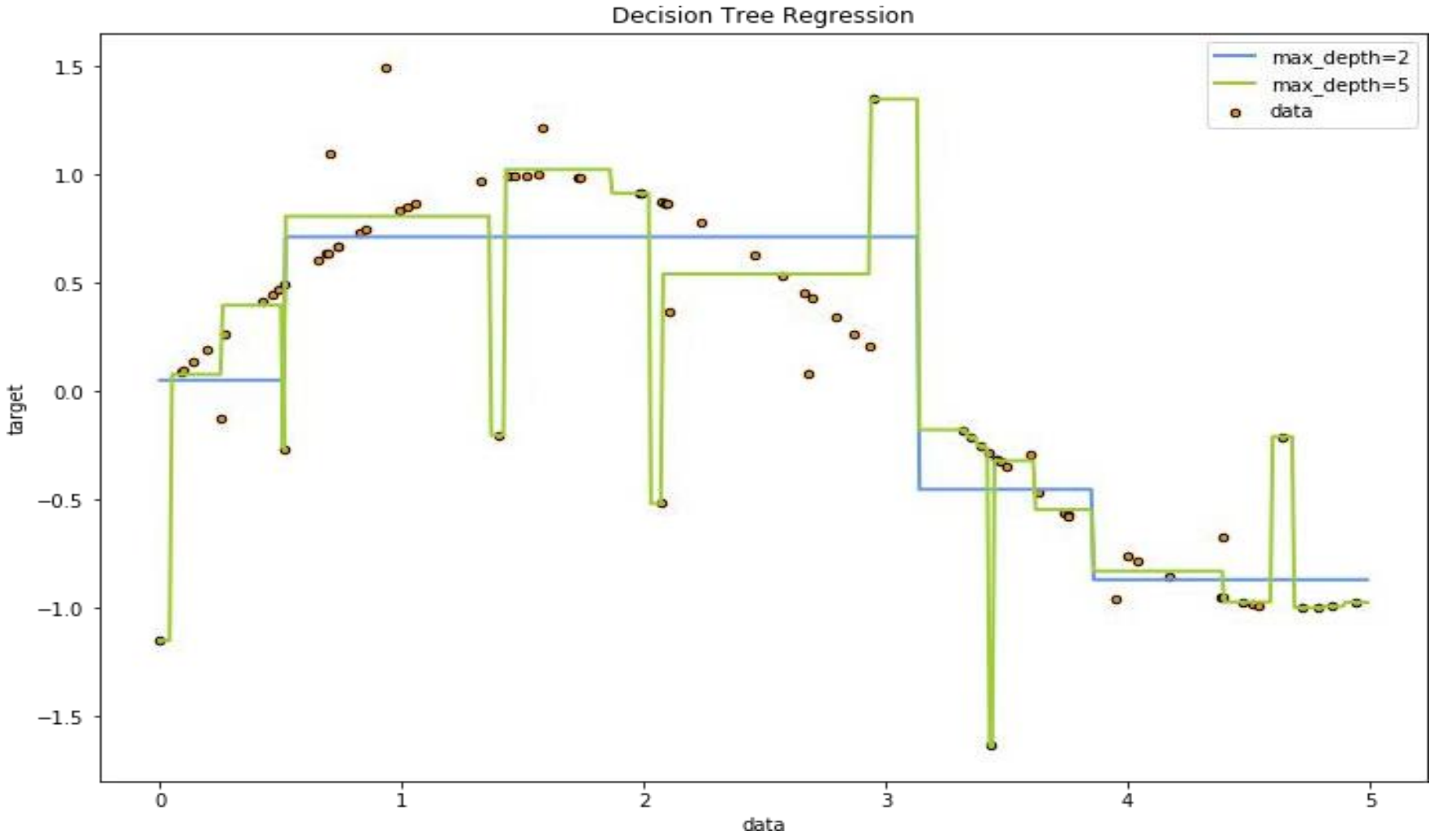
```
import numpy as np
from sklearn.svm import SVR
import matplotlib.pyplot as plt

# Generate sample data
X = np.sort(5 * np.random.rand(40, 1), axis=0)
Y = np.sin(X).ravel()
# Add noise to targets
Y[::5] += 3 * (0.5 - np.random.rand(8))

print("Examples = ",X.shape ," Y = ", Y.shape)
# Fit regression model
svr_rbf  = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=.1)
svr_lin  = SVR(kernel='linear', C=100, gamma='auto')
svr_poly = SVR(kernel='poly', C=100, gamma='auto', degree=3,
epsilon=.1, coef0=1)

# Look at the results
lw = 2
svrs = [svr_rbf, svr_lin, svr_poly]
kernel_label = ['RBF', 'Linear', 'Polynomial']
model_color = ['m', 'c', 'g']
```

```
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 10), sharey=True)
for ix, svr in enumerate(svrs):

    axes[ix].plot(X, svr.fit(X, Y).predict(X), color=model_color[ix], lw=lw,
                  label='{} model'.format(kernel_label[ix]))

    axes[ix].scatter(X[svr.support_], Y[svr.support_], facecolor="none",
                     edgecolor=model_color[ix], s=50,
                     label='{} support vectors'.format(kernel_label[ix]))
    axes[ix].scatter(X[np.setdiff1d(np.arange(len(X)), svr.support_)],
                     Y[np.setdiff1d(np.arange(len(X)), svr.support_)],
                     facecolor="none", edgecolor="k", s=50,
                     label='other training data')
    axes[ix].legend(loc='upper center', bbox_to_anchor=(0.5, 1.1),
                    ncol=1, fancybox=True, shadow=True)

fig.text(0.5, 0.04, 'data', ha='center', va='center')
fig.text(0.06, 0.5, 'target', ha='center', va='center', rotation='vertical')
fig.suptitle("Support Vector Regression", fontsize=14)
plt.show()
```

# 1. Regression: Support Vector Regression SVR



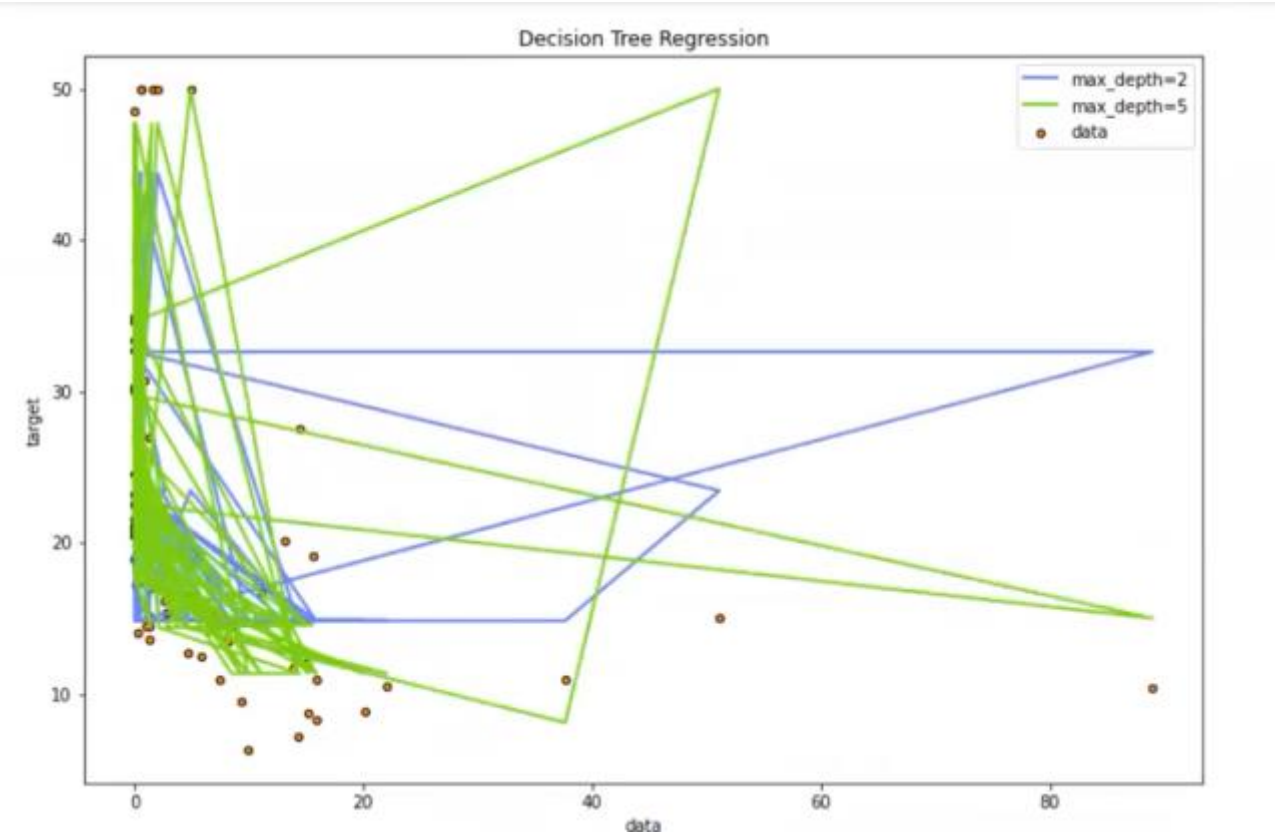
# 1. Regression: Decision Trees Regression



# 1. Regression: Decision Trees Regression

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.tree import DecisionTreeRegressor

rng = np.random.RandomState(1)
X = np.sort(5 * rng.rand(80, 1), axis=0)
Y = np.sin(X).ravel()
Y[::5] += 3 * (0.5 - rng.rand(16))
print("Examples = ",X.shape ," Labels = ", Y.shape)
X_test = np.arange(0.0, 5.0, 0.01)[:, np.newaxis]

regressor1 = DecisionTreeRegressor(max_depth=2)
regressor2 = DecisionTreeRegressor(max_depth=5)
regressor1.fit(X, Y)
regressor2.fit(X, Y)
predicted1 = regressor1.predict(X_test)
predicted2 = regressor2.predict(X_test)

plt.figure(figsize=(12,8))
plt.scatter(X, Y, s=20, edgecolor="black",c="darkorange",
label="data")
plt.plot(X_test, predicted1,
color="cornflowerblue",label="max_depth=2", linewidth=2)
plt.plot(X_test, predicted2, color="yellowgreen",
label="max_depth=5", linewidth=2)
plt.xlabel("data")
plt.ylabel("target")
plt.title("Decision Tree Regression")
plt.legend()
plt.show()
```

# 1. Regression: Decision Trees Regression



# 1. Regression: Random Forest Regressor

```
from sklearn.ensemble import
RandomForestRegressor

regressor =  RandomForestRegressor(max_depth=5)
regressor.fit(X, Y)
predicted = regressor1.predict(X)
```